

GitHub Calendar Sync – Vollständiges technisches Runbook

1. Ziel des Systems (Warum existiert das?)

Dieses System automatisiert die Erstellung einer iCalendar-Datei (.ics) aus einem Python-Skript und veröffentlicht diese regelmäßig über GitHub. Die Datei wird anschließend von Google Calendar abonniert. Wichtig: - Kein eigener Server notwendig - Vollständig automatisiert - Aktualisierung erfolgt zeitgesteuert über GitHub Actions

2. Architektur (wie das System funktioniert)

```
Python Script (Logik)
↓
GitHub Repository (Code + Dateiablage)
↓
GitHub Actions (automatische Ausführung)
↓
GitHub Pages (öffentliche URL)
↓
Google Calendar (liest ICS Datei)
```

Wichtiges Verständnis: GitHub Actions ist kein Server, sondern ein temporärer Runner. Er startet, führt das Skript aus und verschwindet wieder.

3. Schritt 1 – GitHub Konto

Ein GitHub Konto wird benötigt, um: - Code zu speichern - Automatisierungen (Actions) zu nutzen - Datei öffentlich bereitzustellen (Pages)

4. Schritt 2 – Repository erstellen

Repository ist der zentrale Speicherort für: - Python Skript - Workflow Definition - erzeugte Kalenderdatei

Wichtig: Repository muss PUBLIC sein, sonst funktioniert GitHub Pages nicht ohne Einschränkungen.

5. Schritt 3 – Repository klonen

```
git clone https://github.com/USER/REPO.git
cd REPO
```

Was passiert hier? - Git lädt Projekt lokal herunter - du arbeitest auf deinem Rechner - Änderungen werden später zurück hochgeladen

Typischer Fehler: GitHub akzeptiert KEINE Passwörter mehr bei HTTPS Git Zugriff. → Lösung: Personal Access Token verwenden

6. Schritt 4 – Projektstruktur

Benötigte Dateien: - generate_calendar.py → erzeugt ICS Datei - requirements.txt → Python Abhängigkeiten - .github/workflows/update.yml → Automatisierung

7. Schritt 5 – requirements.txt

Diese Datei definiert externe Bibliotheken. Nur Pakete, die NICHT Teil von Python sind.

```
requests
ics
beautifulsoup4
reportlab
```

Fehlerquelle: Wenn hier etwas fehlt → GitHub Action bricht mit "Module not found" ab.

8. Schritt 6 – Python Skript

Das Skript erzeugt die Datei kalender.ics. Wichtig: - jedes Event benötigt stabile UID - Zeit muss korrekt gesetzt sein (UTC empfohlen)

Fehlerquelle: Wenn UID sich ändert → Google Calendar erstellt doppelte Termine.

9. Schritt 7 – GitHub Actions

GitHub Actions führt das Skript automatisch aus.

```
on:
  schedule:
    - cron: '0 */6 * * *'
```

Bedeutung: - alle 6 Stunden - Zeit basiert auf UTC

10. Schritt 8 – Git Befehle (ERKLÄRUNG)

Git arbeitet in 3 Stufen:

1. Arbeitsverzeichnis
→ Datei wurde geändert
2. Staging Area (git add)
→ Änderungen vorgemerkt
3. Commit (git commit)
→ lokaler Versions-Snapshot
4. Remote (git push)
→ Upload zu GitHub

Warum diese Trennung? → Du kannst Änderungen sammeln bevor du sie veröffentlichst → Versionen bleiben nachvollziehbar

11. Schritt 9 – Git Reset (Fehlerkorrektur)

Reset wird genutzt, wenn etwas falsch gespeichert wurde.

```
git reset  
→ entfernt Staging
```

```
git reset HEAD~1  
→ letzten Commit entfernen (Dateien bleiben)
```

```
git reset --hard HEAD~1  
→ alles löschen (gefährlich)
```

Warnung: --hard löscht echte Arbeit unwiederbringlich.

12. Schritt 10 – GitHub Actions Debugging

Fehleranalyse erfolgt im GitHub Actions Tab.

Typische Fehler: - Module not found → requirements.txt fehlt - Authentication failed → Token falsch - Workflow permission error → workflow scope fehlt

13. Schritt 11 – GitHub Pages

Aktivierung: Settings → Pages → Deploy from branch

Ergebnis: Öffentliche URL zur ICS Datei

<https://USER.github.io/REPO/kalender.ics>

14. Schritt 12 – Google Kalender

Google Kalender abonniert die ICS Datei per URL.

Wichtig: - Aktualisierung kann 1–24 Stunden dauern - Google cached externe Kalender

15. Cron Übersicht

0 * * * *	→ jede Stunde
0 */2 * * *	→ alle 2 Stunden
0 */6 * * *	→ alle 6 Stunden
0 5 * * *	→ täglich 05:00 UTC

16. Gesamt-Fehlerdiagnose (WICHTIG)

Wenn etwas nicht funktioniert: 1. Läuft GitHub Action? → sonst kein Update 2. Wird kalender.ics erzeugt? → sonst Python Fehler 3. Ist Pages aktiv? → sonst keine URL 4. Sieht Google alte Daten? → nur Cache Problem